

SLE-4: Final Viva & Architecture Decision Record (ADR)

Course: 02AML204 – Introduction to Artificial Intelligence

PRN	25UAM126
Name	Sanskar Gorave
Division	B
Date	10/10/2026
GitHub	https://github.com/SanskarG-09/SLE--AI.git

PART A — Architecture Decision Record

ADR Title: Use of a Visited Set and Algorithm-Specific Frontier Data Structures in BFS and DFS

Status: Accepted

Date: 10/10/2026

PRN : 25UAM126

Name : Sanskar Gorave

1. Context

My Factory-Floor Search System finds the Quality Inspection Station (G) from the Main Control Room (S) on a 6×6 factory grid. Cells marked # are blocked, and a search can move up, down, left or right. In SLE-2 I compared BFS and DFS on this grid, and in SLE-3 I described the system with the C4 model. While writing the code, I had to decide how to store the cells waiting to be explored (the frontier) and how to avoid visiting the same cell again. On an open grid, neighbouring cells lead back to each other, so a search with no memory can repeat work or loop. The frontier structure also decides whether a search is BFS or DFS, so this choice affects the fairness of the SLE-2 comparison.

2. Decision

In `factory_search.py`, `bfs()` and `dfs()` use the same grid, start, goal, obstacles and four directions, and both keep a **visited set** of (row, col) cells. A cell is added to the set when it is first discovered, before it joins the frontier. **BFS** uses a FIFO queue (`collections.deque` with `popleft()`) and **DFS** uses a LIFO stack (a Python list with `pop()`). Only the frontier differs between the two.

3. Alternatives Considered

- **No visited set:** not selected because cells could be added many times, and DFS could move back and forth between neighbouring cells without ending.
- **A plain list for both searches:** not selected because BFS removes from the front, and `list.pop(0)` shifts every item, while `deque.popleft()` does not.
- **Recursive DFS instead of an explicit stack:** not selected because an explicit stack keeps DFS in the same structure as BFS, so the frontier is the only difference.

4. Rationale / Justification

Metric (SLE-2, same 6×6 grid)	BFS	DFS
Run 1, ms per 10,000 searches	285.9454	145.2534
Run 2, ms per 10,000 searches	207.8587	142.8757
Run 3, ms per 10,000 searches	185.3564	136.0209
Average, ms per 10,000 searches	226.3868	141.3833
Nodes explored	22	17

- **Measured (SLE-2):** DFS was faster in all three runs (141.3833 ms against 226.3868 ms on average) and explored 17 nodes against 22. Fewer nodes means less work, which explains the lower time.
- **Limit of this result:** it holds only for this grid and this order. The code pushes neighbours as up, down, left, right, so DFS tries the right-hand neighbour first. I do not claim DFS is generally faster. BFS times also varied between runs (285.9454 to 185.3564 ms).

- **Theory (not measured):** BFS finds a shortest path in an unweighted grid, and DFS does not guarantee this. My code returns only the node count, so path length was not measured.
- **Why these structures:** the queue explores cells in order of distance from S (BFS), and the stack goes deep along one route first (DFS). Both `popLeft()` and `pop()` take constant time. The visited set lets each open cell enter the frontier once, so the search finishes and the work is limited by the 36 cells.
- **Link to SLE-3:** the Container diagram has a Search Engine and a separate Memory / Visited Set, and the Component diagram shows the Frontier as the only difference between BFS and DFS. The Code Level lists `bfs()` and `dfs()`. This decision is what makes the comparison fair.

5. Consequences

Positive:

- Each open cell is explored at most once, so both searches stop on this grid.
- BFS and DFS differ only in the frontier, so the comparison is fair and easy to explain.
- BFS keeps its shortest-path guarantee, while DFS was faster on this grid.

Negative / Trade-offs:

- The visited set and the BFS frontier use more memory as the grid grows.
- DFS has no shortest-path guarantee, and its good result here depends on the layout and direction order.
- The code reports only nodes explored, not the route found, and weighted costs (for example A*) would need a different design.

6. References

- GitHub repository: <https://github.com/SanskarG-09/SLE--AI.git>
- SLE-2 Profiling Report (20/09/2026): Section 3 Results and Section 4 Justification & Analysis.
- SLE-3 Full C4 Model report (04/10/2026): Container, Component and Code Level diagrams, and Section 6 Design Decisions.
- `factory_search.py`: the BFS/DFS code and timing code checked for this ADR.

PART B — Short Summary of My SLE Journey

- **SLE-1:** NexaBot AI is a simple rule-based Python chatbot. Its README shows I practised Python basics, debugging, Git/GitHub and AI-assisted coding. It is a separate project from the factory-floor search.
- **SLE-2:** I profiled BFS and DFS on a 6×6 factory grid with `py-spy` and `perf_counter()` (3 runs of 10,000 searches). DFS averaged 141.3833 ms against 226.3868 ms for BFS and explored 17 nodes against 22.
- **SLE-3:** I described the same system at four C4 levels: Context, Container, Component and Code.
- **SLE-4:** I wrote this ADR on the visited set and the queue/stack frontiers, justified by the code, the SLE-2 numbers and the SLE-3 architecture.

PART C — AI Contribution Note

- **AI tools used:** Claude for this SLE-4 document. My earlier reports list ChatGPT (SLE-2) and ChatGPT and Claude (SLE-3).
- **What AI helped with:** For SLE-4, Claude helped organise the document in the faculty structure, draft the wording from my SLE-1 to SLE-3 files and `factory_search.py`, and check the figures. In earlier SLEs, AI helped with the BFS/DFS code, `py-spy` explanations and the C4 structure and draft text, as my reports state.
- **What I did myself:** I ran the SLE-2 experiment in VS Code, generated the `py-spy` flame graphs, and collected the performance results. For SLE-3, I selected the SLE-2 system as the base, ran `factory_search.py`, and exported the diagrams from draw.io. For SLE-4, I reviewed the ADR against my BFS and DFS project, checked the performance results and C4 architecture, and edited the document to match the faculty guidelines.